

Verification of a Configurable Vending Machine Controller

Reza

February 12, 2026

Contents

1	Verification Environment and Reproducibility	2
2	Project files hierarchy	2
3	System Specifications and Design (Part 1)	2
3.1	RTL Implementation and FSM Logic	2
3.2	Functional Testing (Simple Testbench)	3
3.3	Constrained Random Verification	4
4	Assertion-Based Verification (Part 2)	5
4.1	Temporal Property Definitions	7
4.2	Checker Binding and Simulation	7
5	Assertion Mining (Part 3)	7
5.1	Configuration and Results	7
5.2	Mining Results and Correlation	7

1 Verification Environment and Reproducibility

For making development and test, I have tried to create an Environment based on Docker image that is combination of configuration of vsim and HARM tools. The tools is available on a Github repository.

<https://github.com/MrzAhmadi/SVT-Environment>

2 Project files hierarchy

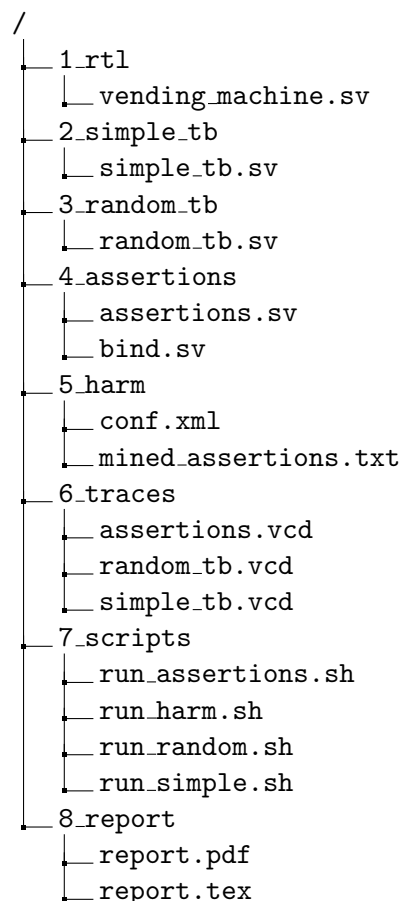


Figure 1: Project Directory Structure

3 System Specifications and Design (Part 1)

3.1 RTL Implementation and FSM Logic

The main purpose of this section is design a finite state machine (FSM) to manage the vending machine lifecycle. The main design file is located at `/1_rtl/vending_machine.sv`. This implementation covers all demanded features of vending machine in the part of 1.1 of assignment.

State Transition Table

The following table describes the behavior of the FSM across its three primary states:

Current State	Next State	Condition / Action
IDLE	IDLE	Accumulating credit via <code>coin_in</code> . Valid coins: 10, 20, 50, 100, 200.
IDLE	DELIVER	<code>button_in</code> pressed AND <code>credit</code> $\geq$ cost (30 for Water, 50 for Soda). Subtract cost from credit.
DELIVER	CHANGE	Timer reaching N cycles. Assert <code>beverage_out</code> . Transition if <code>credit</code> $\neq 0$ and cannot buy more items.
DELIVER	IDLE	Timer reaching N cycles. Assert <code>beverage_out</code> . Transition if <code>credit</code> $== 0$.
CHANGE	IDLE	Timer reaching M cycles. Assert <code>change_out</code> and reset credit to 0.

Table 1: FSM State transitions and associated logic for the Vending Machine.

3.2 Functional Testing (Simple Testbench)

In this part we describe the manual test cases from Part 1.2 of assignment. The file that is located at `2_simple_tb/simple_tb.sv`. In this simple testbench we consider some scenarios to test on our FSM vending machine. First of all we initialize system in 20ns period then we try to test some cases in this order:

- Water Purchase (Exact Change): Inserts 30 cents (10 + 20) and select water (`button1`)
- Water Delivery Timing: Trying to verify N cycles after press the button
- Soda Purchase (with Change): Inserts 60 cents (50 + 10) and select soda (`button2`)
- Change Return Logic: Confirming that machine returns remaining 10 cents
- Change Return Timing: Trying to verify M cycles after delivery.

For running this test bench, we have dedicated a bash script that is located at `7_scripts/run_simple.sh`. This script compiles the vending_machine FSM and the simple testbench using `vlog`, and then runs `vsim` to create traces and save them at `6_traces/simple_tb.vcd`.

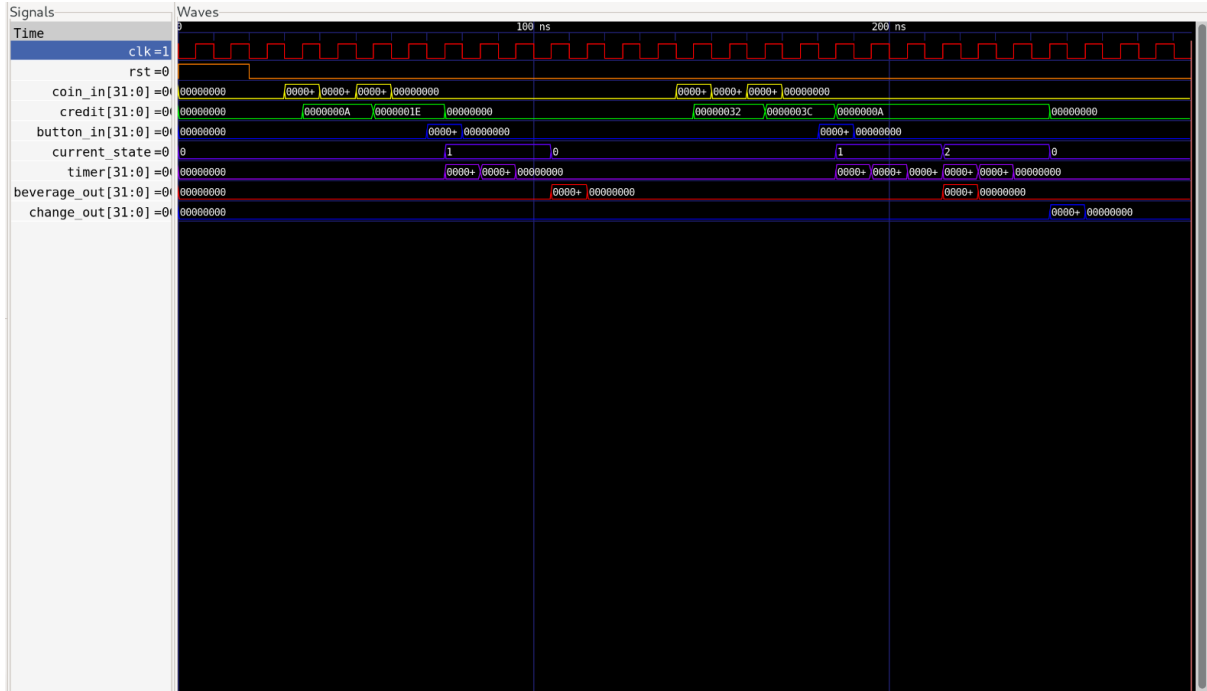


Figure 2: Simulation trace of the simple testbench (Part 1.2). The waveform demonstrates a successful Water purchase (0-120ns) followed by a Soda purchase with change return (150-280ns), confirming the FSM transitions and N/M cycle delays.

3.3 Constrained Random Verification

This is an implementation for Part 1.3. The random testbench is located at `random_tb/random_tb.sv`. It uses a generator and a transition class automatically create 2,000 cycles of test data. There were some efforts to apply realistic distribution to mimic real-world human. There are some implementation for HARM mining. There is also another bash script at `7_scripts/run_random.sh` that runs this random testbench and saves traces at `6_traces/random_tb.vcd`.

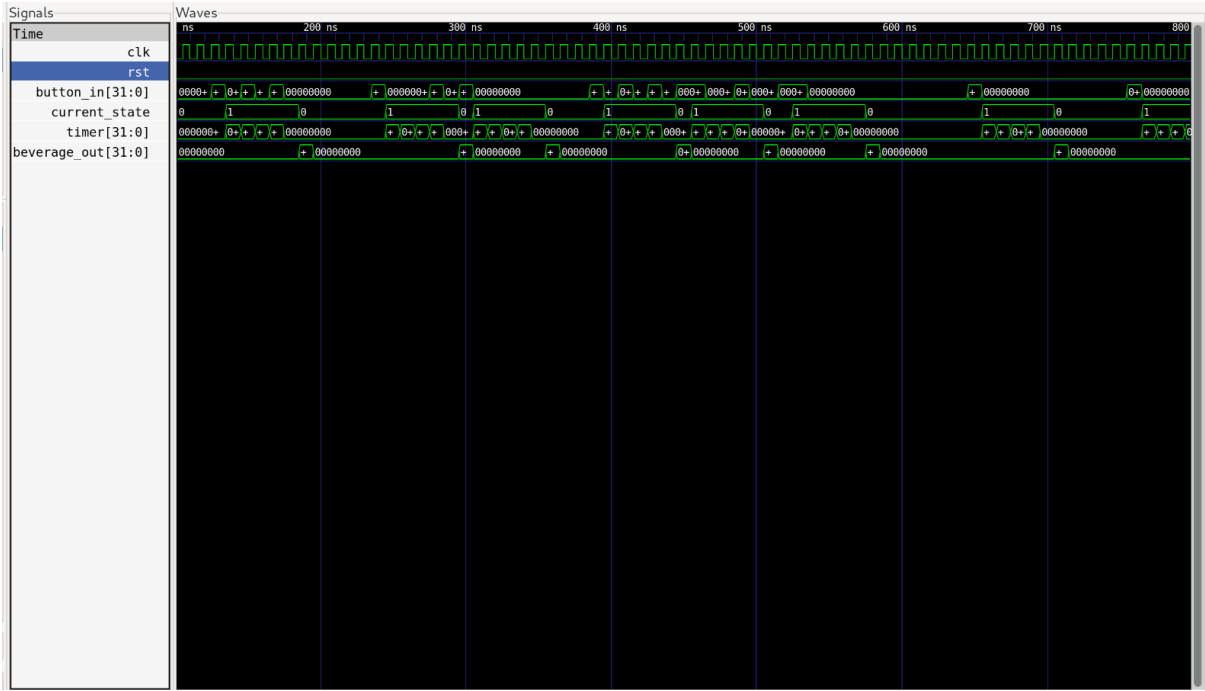


Figure 4: Visual verification of the `a_water_delivery` assertion over multiple transactions. The waveform confirms that every time the FSM enters the DELIVER state (1), the timer counts correctly, and the beverage output is asserted after the delay, verifying the property holds consistently.

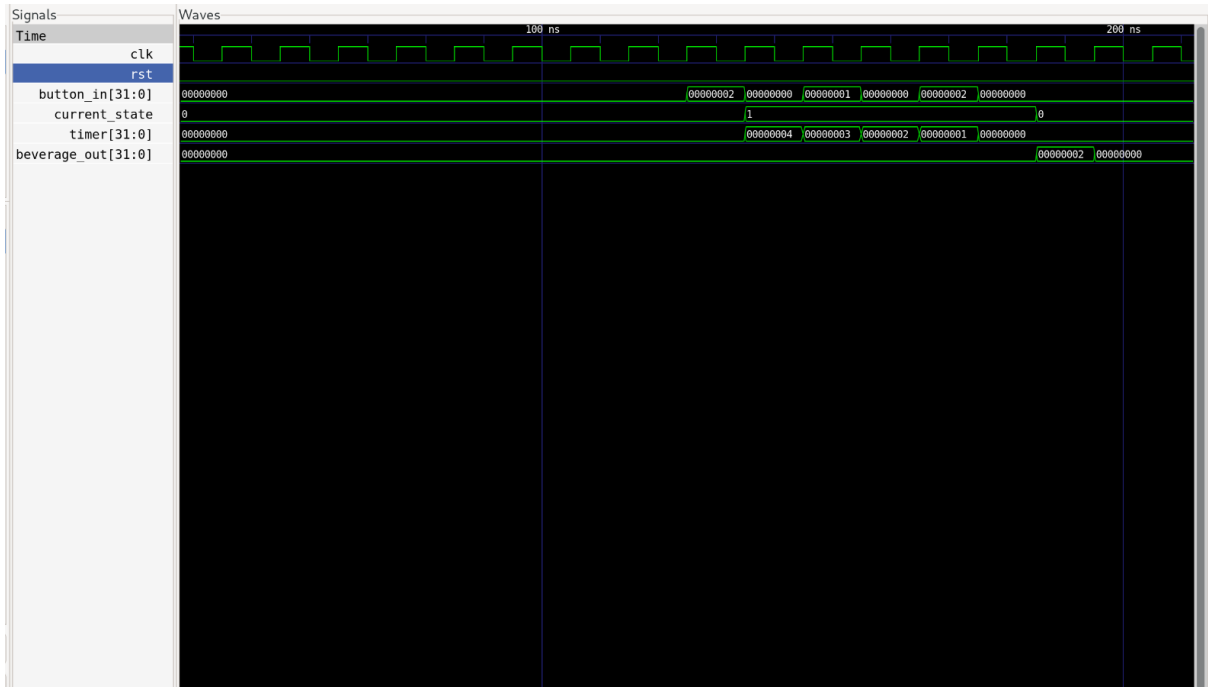


Figure 5: Detailed visual verification of the `a_soda_delivery` assertion. The trace shows a Soda request (`button_in=2`), followed by the timer counting down exactly $N = 4$ cycles while in the DELIVER state (1), before asserting `beverage_out`.

4.1 Temporal Property Definitions

For Part 2.1, five temporal assertions were developed in SystemVerilog (SVA) within the `vending_props.sv` file. These properties monitor the FSM:

- **Reset Safety (`a_reset`):** Verifies that when the reset signal is high, the internal credit, beverage output, and change output are all forced to zero.
- **Credit Integrity (`a_coin_inc`):** Ensures that in the IDLE state, the credit register correctly increments using the `$past` function.
- **Water Delivery Timing (`a_water_delivery`):** Checks that beverage output is asserted exactly N clock cycles later.

4.2 Checker Binding and Simulation

A separate file `4_assertions/bind.sv` was created to attach the `vending_props` checker to the `vending_machine` module.

To run this simulation, the script `7_scripts/run_assertions.sh` is used. This script compiles the design and executes the simulation with the `-assertdebug` flag then we run another bash script at `7_scripts/run_harm.sh`. At the end it saves traces at `6_traces/assertions.vcd`.

5 Assertion Mining (Part 3)

The HARM tool was used to automatically discover design properties from the execution traces.

5.1 Configuration and Results

The `conf.xml` defines manual templates for delivery timing and uses numeric clustering to identify signal patterns. HARM analyzed 2,000 cycles of data and successfully mined the FSM logic, which is documented in `5_harm/mined_assertions.txt`. These results confirm that the design satisfies the temporal requirements discovered during simulation.

5.2 Mining Results and Correlation

N	Assertion (Context : default)	final	causality	frequency
0	$G(\{!(\text{harm_bev}==32'b1) \ \&\& \ !(\text{harm_bev}==32'b10)\} \rightarrow \text{harm_bev}==32'b0)$	1.00	1.00	1.00
1	$G(\{!(\text{harm_btn}==32'b1) \ \&\& \ !(\text{harm_btn}==32'b10)\} \rightarrow \text{harm_btn}==32'b0)$	1.00	1.00	0.90
2	$G(\{!(\text{harm_btn}==32'b0) \ \&\& \ !(\text{harm_btn}==32'b10)\} \rightarrow \text{harm_btn}==32'b1)$	0.00	1.00	0.11
3	$G(\{!(\text{harm_btn}==32'b0) \ \&\& \ !(\text{harm_btn}==32'b1)\} \rightarrow \text{harm_btn}==32'b10)$	0.00	1.00	0.10
4	$G(\{!(\text{harm_bev}==32'b0) \ \&\& \ !(\text{harm_bev}==32'b10)\} \rightarrow \text{harm_bev}==32'b1)$	0.00	1.00	0.06
5	$G(\{!(\text{harm_bev}==32'b0) \ \&\& \ !(\text{harm_bev}==32'b1)\} \rightarrow \text{harm_bev}==32'b10)$	0.00	1.00	0.05
6	$G(\{!(\text{harm_bev}==32'b0) \ \#\#3 \ \text{true}\} \rightarrow \text{harm_bev}==32'b0)$	0.00	0.09	0.10
10	$G(\{\text{rst} \ \#\#4 \ \text{true}\} \rightarrow \text{harm_btn}==32'b0)$	0.00	0.19	0.00
12	$G(\{\text{rst}\} \rightarrow \text{harm_btn}==32'b0)$	0.00	0.19	0.00
15	$G(\{\text{harm_bev}==32'b1 \ \#\#4 \ \text{true}\} \rightarrow \text{harm_bev}==32'b0)$	0.00	0.09	0.06
24	$G(\{\text{rst}\} \rightarrow \text{harm_bev}==32'b0)$	0.00	0.09	0.00

Table 2: Mined assertions from HARM output.